Here, the new part is the `HNSWLib` import that uses the `fromDocuments` method to convert the processed text chunks into a vector representation, utilizing the predefined embedding model. This vector store can then be used for semantic search, enabling the RAG system to retrieve relevant information based on user queries.

To perform the indexing of documents effectively, it is best to conduct this process offline, especially when dealing with a large volume of documents. For convenience, a script has been provided that you can run alongside a sample list of documents located in the scripts folder within the `rag-web-app` example.

To index the documents, you must first navigate to the ch07/rag-web-app directory and then execute the following command in your terminal:

```
$ cd ch07/rag-web-app
$ node scripts/indexDocuments.google.js -d corpus
```

This command will initiate the indexing process using the specified corpus of documents. The script is designed to handle the indexing efficiently, ensuring that all documents are processed and stored correctly. I've prepopulated this folder with example documents from the KG-RAG datasets repository (https://github.com/docugami/KG-RAG-datasets) containing several aviation accident reports in PDF form, but feel free to use your own set of data instead.

> **NOTE** This script uses Google Gemini embeddings and requires an active Google Gemini API key to generate vector representations of documents. The API key must be set in the .env file as `GEMINI_API_KEY`.

The script will process your documents, creating vector embeddings that enable semantic search capabilities. Be aware that running this script will incur costs associated with OpenAI's API usage for embedding generation.

If you prefer to use an alternative embedding provider like OpenAI, the companion script `indexDocuments.openai.js` is available, which requires a `OpenAI_API_KEY` instead.

Retrieval

The retrieval process is a critical component of the RAG system, responsible for finding the most relevant documents in response to a user's query. In our implementation, the retrieval uses LangChain's `asRetriever` method, which we explored in the previous chapter.

When loading a previously indexed vector store, the system creates a retriever instance that can efficiently search through the semantic vector representations. In this implementation, the retriever is configured to return the top six most similar document chunks, providing a comprehensive yet focused set of contextually relevant information.

The `asRetriever` method essentially transforms our vector store into a sophisticated search engine. Here is the code snippet that provides this feature: